esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

MODULE AWD

METEOWATCH

4SSA15-G5

Plateforme de surveillance météo et d'alertes en architecture microservices

Encadré par :
Mr. Alaa Rami

Sommaire

01 L'équipe UniDev

02 Le projet

03 Fonctionnalités

04 Architecture & données

05 Microservices

06 Service commun

07 Organisation

Organigramme de l'équipe UniDev

Groupe PostgreSQL

Maha Hammami

Stations

Linda Nebily

Alertes

Chaima Amri

Abonnements

Groupe MongoDB

Ramzi Ben Hmida

Relevés

Amine Arfaoui

Prévisions

Mohamed Taher Elkhouja

Notifications

METEOSPHERE

Idée & Problématique

Les alertes météo arrivent souvent trop tard, dispersées entre plusieurs sources, et ne sont pas adaptées à la région de chaque citoyen.

01

Objectif : suivre la météo en temps réel par station et par ville, et déclencher des vigilances (jaune, orange, rouge).

> ***MeteoWatch centralise les relevés des stations, les prévisions et les alertes, puis prévient automatiquement les personnes concernées.***

02

Valeur : des notifications ciblées par abonnement et une architecture évolutive en microservices.

METEOSPHERE

Les acteurs du système

Trois profils, trois niveaux d'accès gérés par Keycloak

Administrateur

Gère les stations et capteurs, les règles d'alerte et les comptes utilisateurs (Keycloak).

Rôle Keycloak : ADMIN

Météorologue

Saisit et valide les relevés, consulte les prévisions, déclenche et clôture les vigilances.

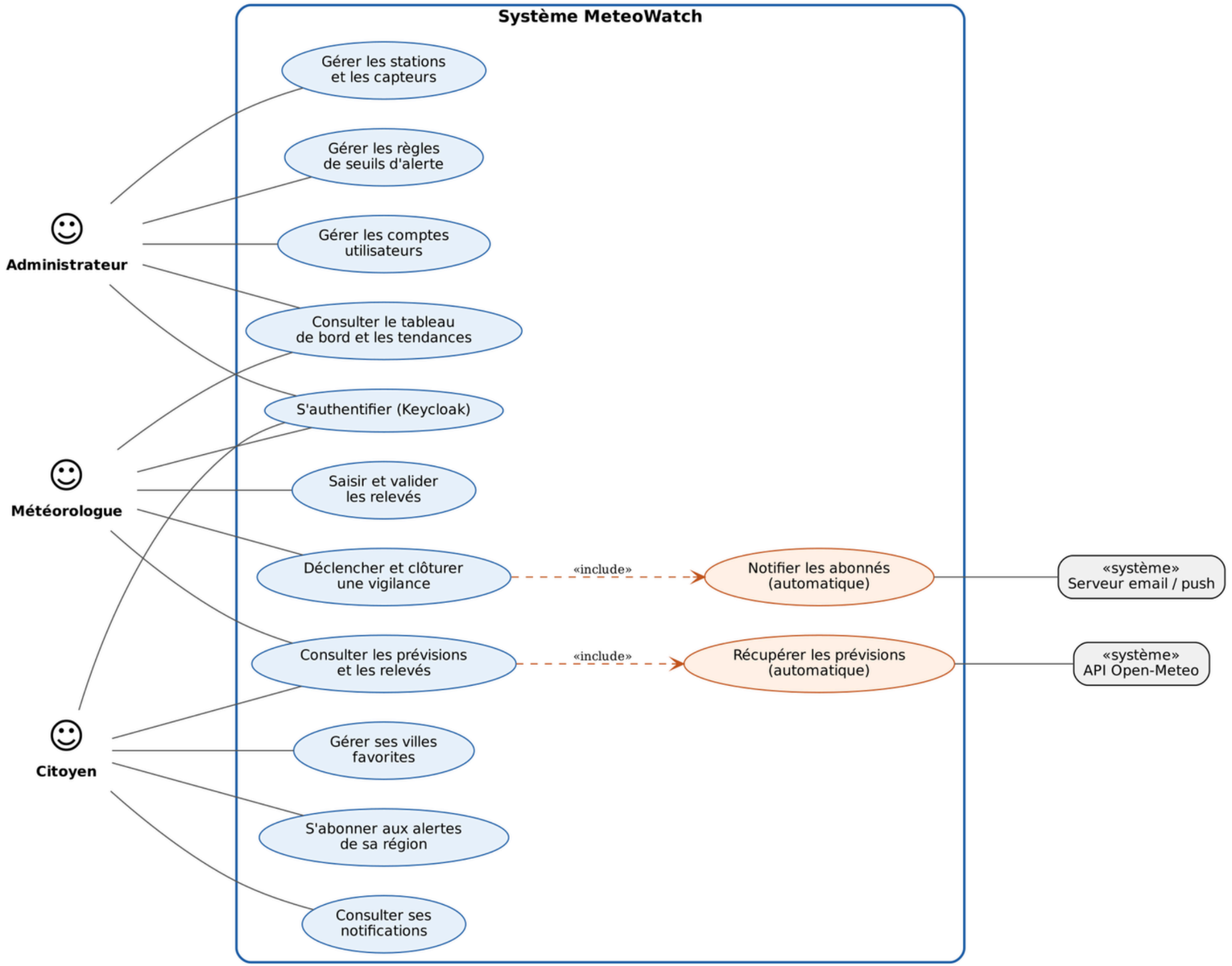
Rôle Keycloak : METEOROLOGUE

Utilisateur (citoyen)

Consulte la météo de ses villes favorites, s'abonne aux alertes de sa région et reçoit les notifications.

Rôle Keycloak : USER

MeteoWatch – Diagramme de cas d'utilisation général



METEOSPHERE

Fonctionnalités principales

Stations & relevés

Gestion des stations et capteurs, enregistrement des mesures (température, humidité, vent, pluie).

Alertes & vigilances

Seuils configurables et déclenchement automatique des vigilances (jaune, orange, rouge) par région.

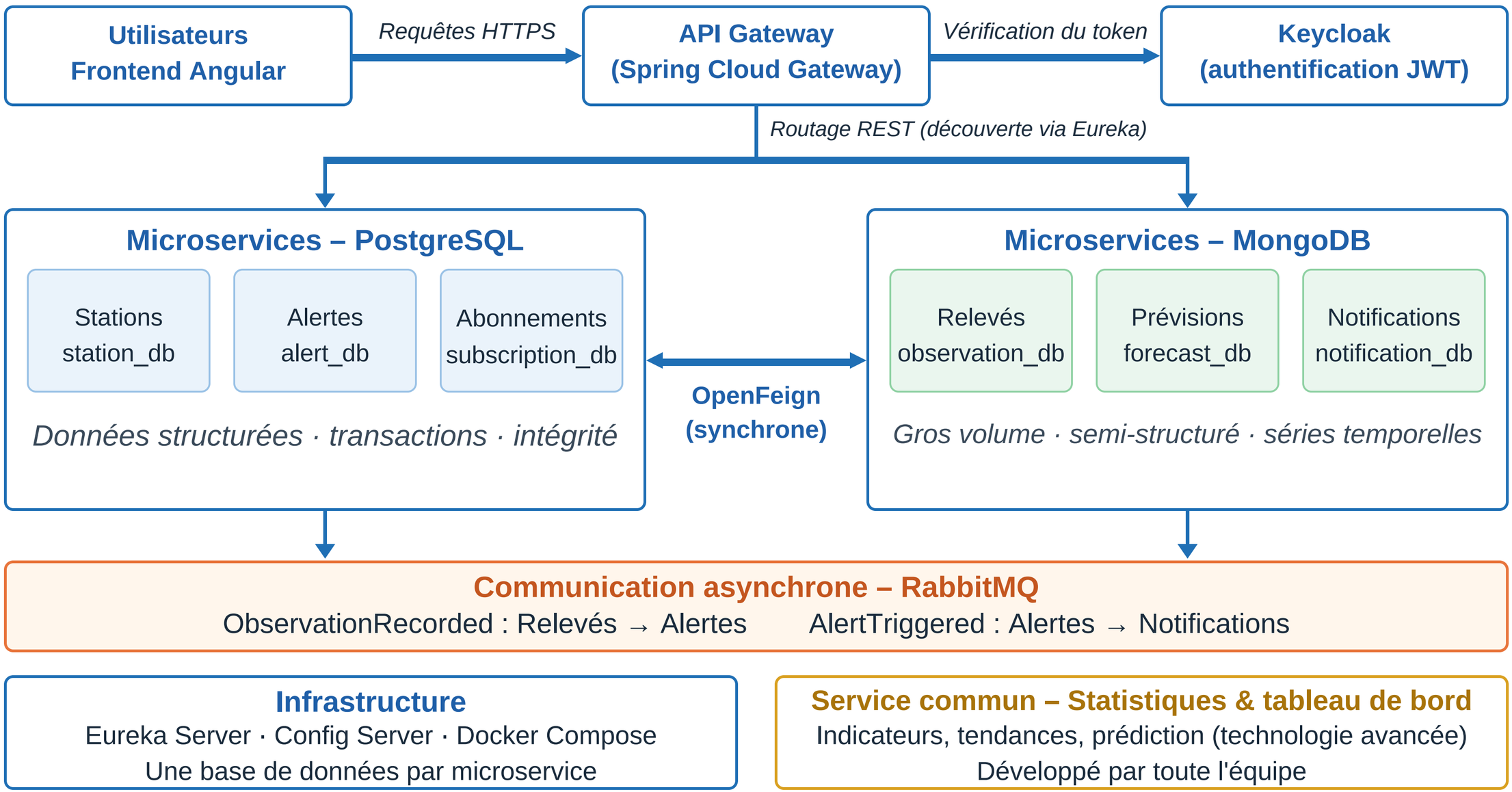
Prévisions

Prévisions sur 7 jours par ville, récupérées d'une API externe et comparées aux relevés réels.

Abonnements & notifications

Villes favorites, abonnement aux alertes par région et notifications email / push avec historique.

Architecture globale de la solution



METEOSPHERE

Microservices & bases de données

Une base de données par microservice — aucune base partagée

PostgreSQL

3 MICROSERVICES

Stations — station_db

Alertes — alert_db

Abonnements — subscription_db

Données structurées et relationnelles, cohérence et transactions (ACID).

MongoDB

3 MICROSERVICES

Relevés — observation_db

Prévisions — forecast_db

Notifications — notification_db

Gros volume, données semi-structurées et séries temporelles, schéma flexible.

Service commun

TECHNOLOGIE AVANCÉE

Statistiques &

tableau de bord

stats_db

Prédiction ou assistant IA, développé par toute l'équipe.

METEOSPHERE

Communication entre services

Comment les microservices échangent entre eux

Synchrone

OPENFEIGN · REST

Relevés → Stations

Prévisions → Stations

Notifications → Abonnements

Réponse immédiate attendue

Asynchrone

RABBITMQ · ÉVÉNEMENTS

Relevés → Alertes

Alertes → Notifications

ObservationRecorded · AlertTriggered

Résilience

RESILIENCE4J

Circuit breaker

Fallback

Retry + file d'attente

Le système reste disponible

Répartition des tâches

MEMBRE	MICROSERVICE	BASE	TÂCHE TRANSVERSE
Maha Hammami	Stations	**PostgreSQL**	Squelette du Frontend
Linda Nebily	Alertes	**PostgreSQL**	Frontend et maquettes
Chaima Amri	Abonnements	**PostgreSQL**	Keycloak (rôles)
Ramzi Ben Hmida	Relevés	**MongoDB**	API Gateway + Eureka
Amine Arfaoui	Prévisions	**MongoDB**	Config Server + Docker
Mohamed Taher Elkhouja	Notifications	**MongoDB**	README + doc technique

Toute l'équipe : service commun, tests, revues de code (pull requests) et suivi des tâches sur Git

MICROSERVICE 1 · POSTGRESQL

Gestion des stations météo

Responsable : Maha Hammamia

Entités (diagramme de classes)

Station : id, nom, ville, latitude, longitude, altitude, dateInstallation

Capteur : id, type (température, humidité, vent…), unité, état (ACTIF / EN_PANNE)

Relation : une Station possède plusieurs Capteurs (1..*)

Contrat d'API REST

GET /api/stations → 200
GET /api/stations/{id} → 200 / 404
GET /api/stations?ville=Tunis → 200
POST /api/stations → 201 / 400
PUT /api/stations/{id} → 200 / 404
DELETE /api/stations/{id} → 204 / 404
POST /api/stations/{id}/capteurs → 201
PATCH /api/capteurs/{id}/etat → 200

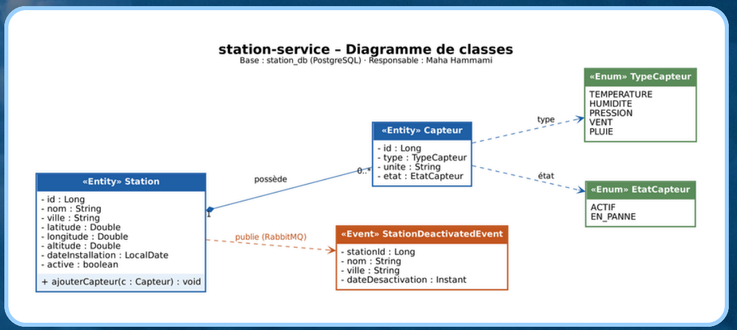
Interactions

Appelé par (OpenFeign, synchrone) :
– Relevés : vérifier qu'une station existe
– Prévisions : récupérer les coordonnées

Événement publié (RabbitMQ) : StationDeactivated

Si indisponible : les appelants passent par un fallback (circuit breaker).

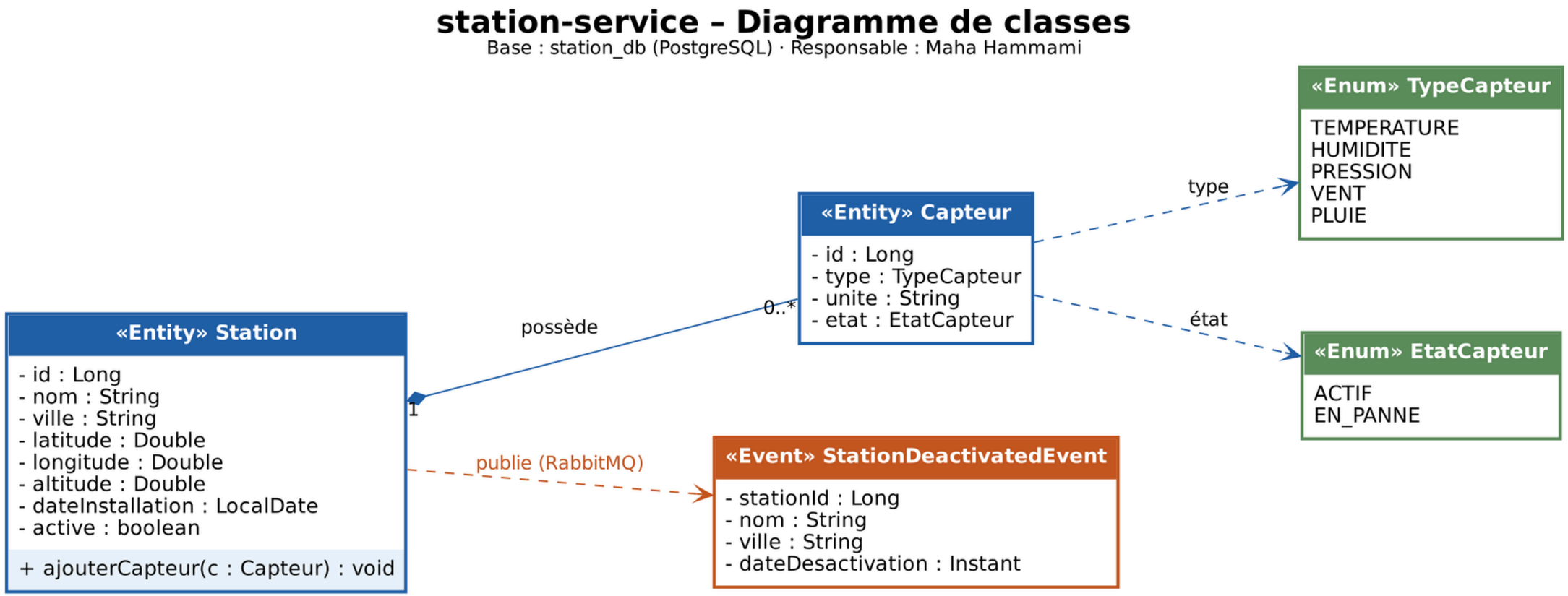
Pourquoi PostgreSQL ? Données structurées et liées (station ↔ capteurs), intégrité référentielle et transactions.



🔍 **Cliquer pour agrandir**

Diagramme de classes – station-service

← Retour



MICROSERVICE 2 · POSTGRESQL

Gestion des alertes et vigilances

Responsable : Linda Nebily

Entités (diagramme de classes)

RegleAlerte : id, type (CANICULE, TEMPETE, PLUIE…), seuil, opérateur, niveau

Alerte : id, region, stationId, niveau (JAUNE / ORANGE / ROUGE), message, statut, dateDebut, dateFin

Relation : une Règle déclenche plusieurs Alertes

Contrat d'API REST

GET /api/alertes?region=&statut= → 200
GET /api/alertes/{id} → 200 / 404
POST /api/alertes → 201 / 400
PATCH /api/alertes/{id}/cloture → 200 / 409
GET /api/regles → 200
POST /api/regles → 201 / 400
PUT /api/regles/{id} → 200 / 404
DELETE /api/regles/{id} → 204 / 404

Interactions

Consomme (RabbitMQ) : ObservationRecorded → compare la valeur aux seuils

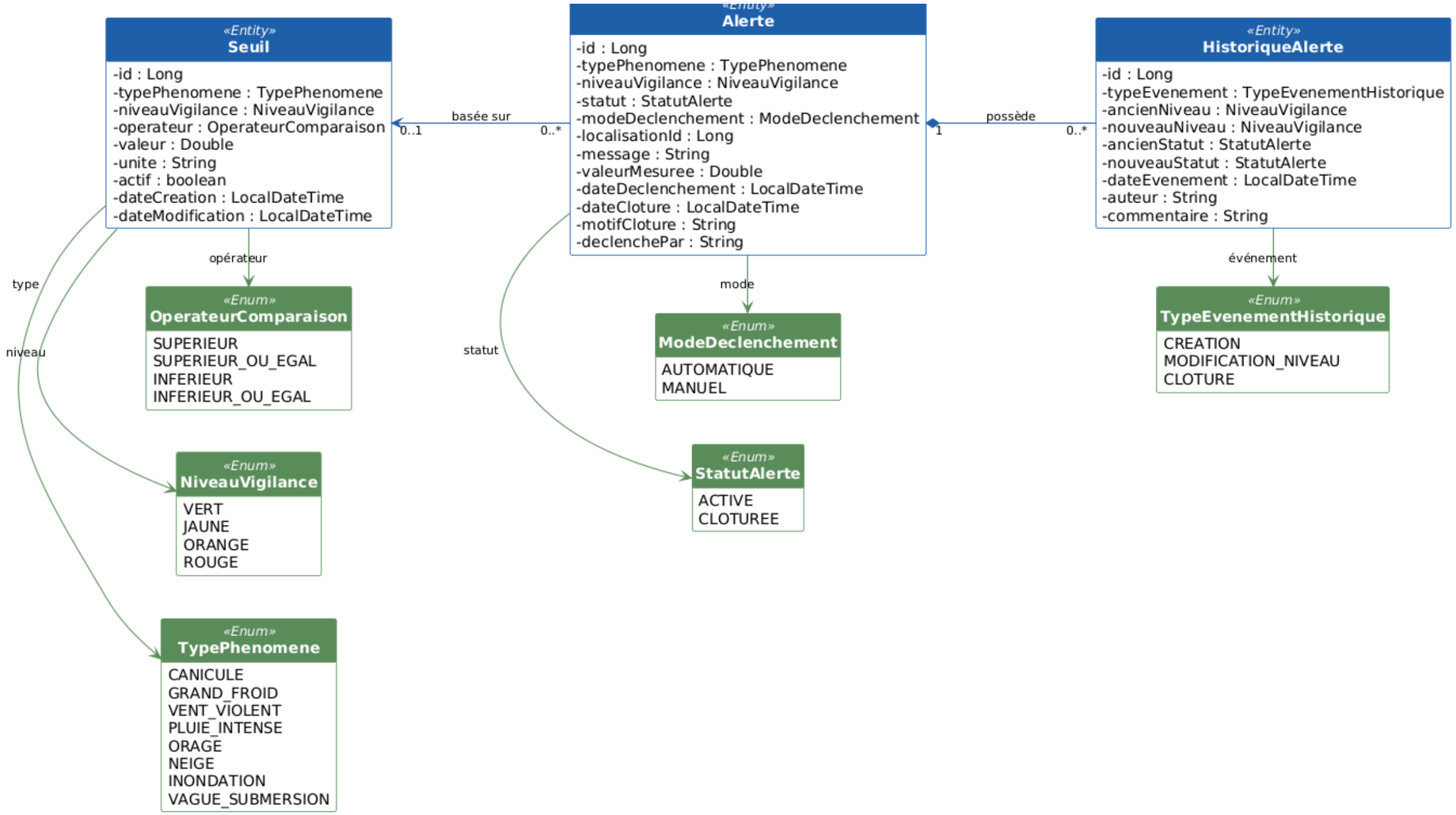
Publie (RabbitMQ) : AlertTriggered {alerteId, region, niveau, message}

Si Notifications est indisponible : le message reste dans la file.

Pourquoi PostgreSQL ? Règles et alertes structurées, changements de statut cohérents grâce aux transactions.

Diagramme de classes – Alerte service

← Retour



MICROSERVICE 3 · POSTGRESQL

Gestion des utilisateurs et abonnements

Responsable : Chaima Amri

Entités (diagramme de classes)

Utilisateur : id, keycloakId, nom, email, téléphone

Abonnement : id, utilisateurId, region, niveauMin, canal (EMAIL / PUSH), actif

VilleFavorite : id, ville, utilisateurId

Un Utilisateur a plusieurs Abonnements et Villes favorites

Contrat d'API REST

GET /api/utilisateurs/me → 200 / 401
PUT /api/utilisateurs/me → 200 / 400
GET /api/abonnements → 200
POST /api/abonnements → 201 / 409
DELETE /api/abonnements/{id} → 204 / 404
GET /api/abonnements/region/{r} → 200
POST /api/favoris → 201
DELETE /api/favoris/{id} → 204

Interactions

Appelé par (OpenFeign) : Notifications → liste des abonnés d'une région

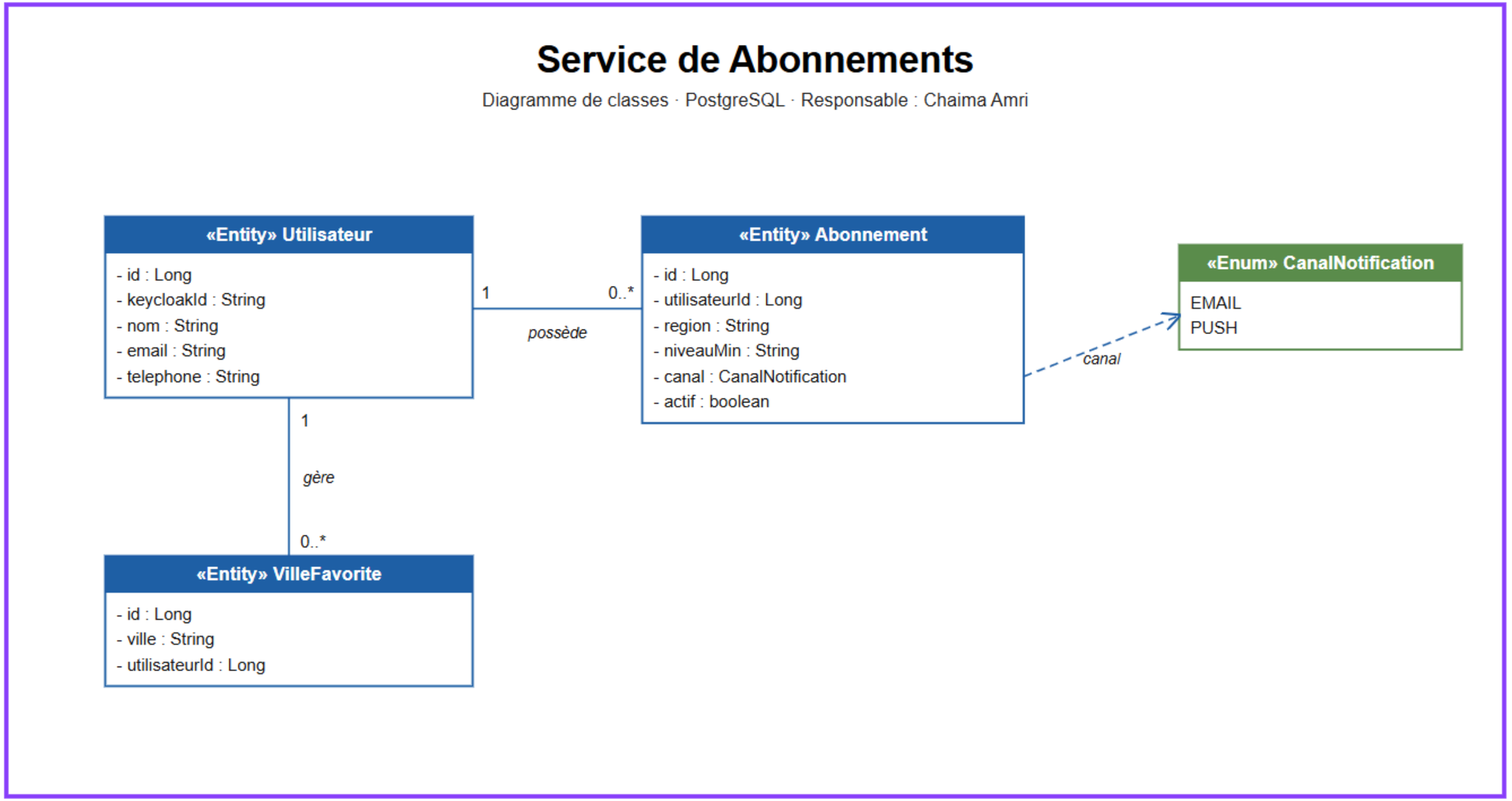
Identité fournie par Keycloak (keycloakId dans le JWT)

Si indisponible : Notifications met l'envoi en attente et réessaie (retry).

Pourquoi PostgreSQL ? Relations utilisateur ↔ abonnements et contraintes d'unicité (pas d'abonnement en double).

Diagramme de classes – Abonnement service

← Retour



MICROSERVICE 4 · MONGODB

Gestion des relevés météo

Responsable : Ramzi Ben Hmida

Documents (modèle de données)

Releve : id, stationId, horodatage, temperature, humidite, pression, vitesseVent, precipitation

StatJournaliere : stationId, date, min, max, moyenne

Collection time-series indexée sur (stationId, horodatage)

Contrat d'API REST

POST /api/releves → 201 / 400 / 404

GET /api/releves?stationId=&from=&to= → 200

GET /api/releves/derniers?stationId= → 200

GET /api/releves/stats?stationId=&date= → 200

DELETE /api/releves/{id} → 204 / 404

404 si la station n'existe pas

Interactions

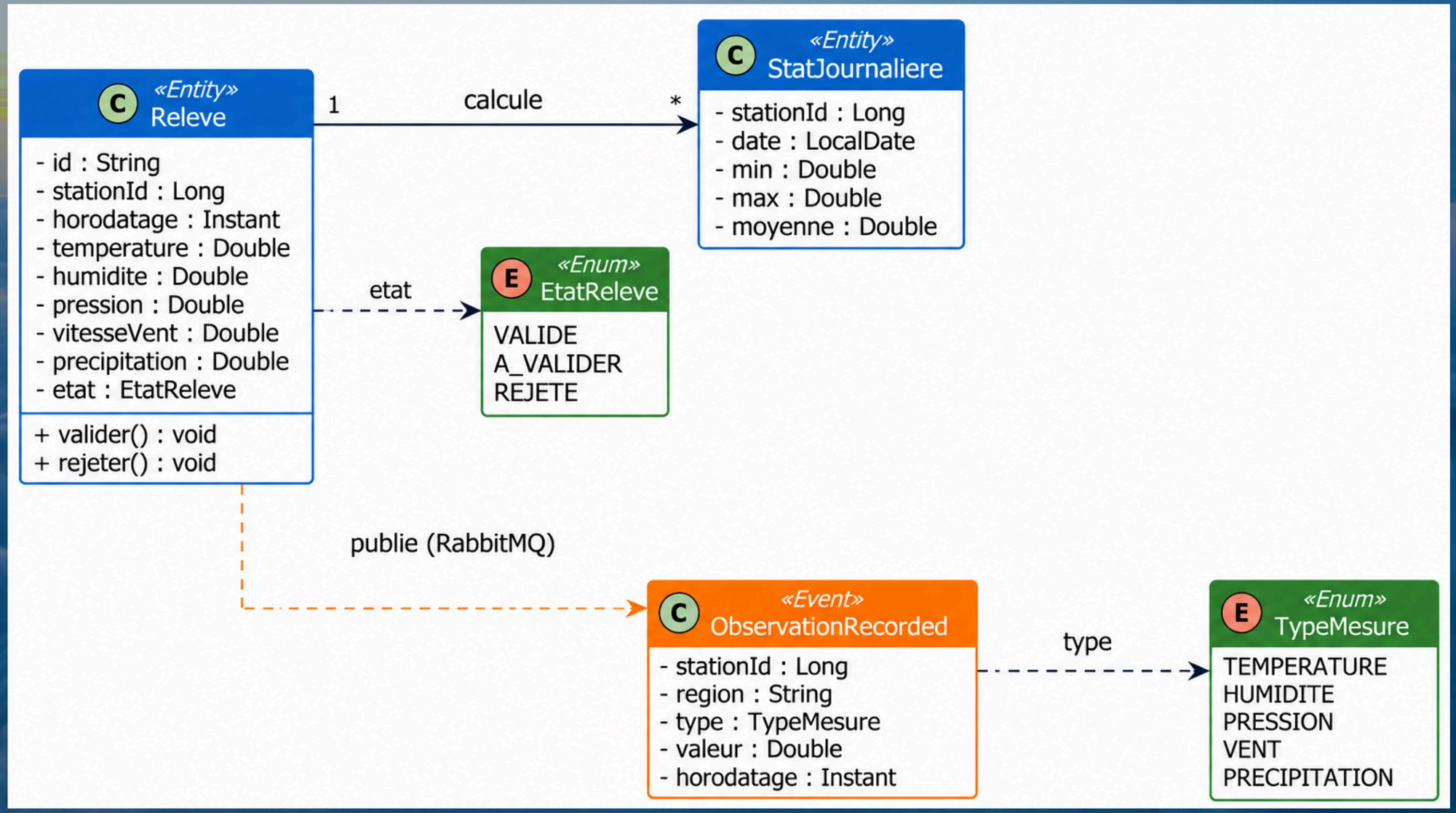
Appelle (OpenFeign) : Stations → GET /api/stations/{id}

Publie (RabbitMQ) : ObservationRecorded {stationId, region, type, valeur}

Si Stations est indisponible : relevé enregistré « à valider » puis revérifié.

Pourquoi MongoDB ? Gros volume d'écritures, format flexible selon les capteurs, collections time-series.

Diagramme de classes – readings-service



MICROSERVICE 5 · MONGODB

Gestion des prévisions

Responsable : Amine Arfaoui

Documents (modèle de données)

Prevision : id, ville, latitude, longitude, dateGeneration, source

Sous-documents jours[] : date, tempMin, tempMax, probaPluie, vent, description

Prévisions mises en cache et rafraîchies régulièrement

Contrat d'API REST

GET /api/previsions?ville=Tunis → 200 / 404
GET /api/previsions/{id} → 200 / 404
POST /api/previsions/refresh?ville= → 202
GET /api/previsions/comparaison → 200
DELETE /api/previsions/{id} → 204 / 404

202 : rafraîchissement lancé en arrière-plan

Interactions

Appelle (OpenFeign) : Stations → coordonnées d'une ville

Appelle (HTTP) : API externe Open-Meteo

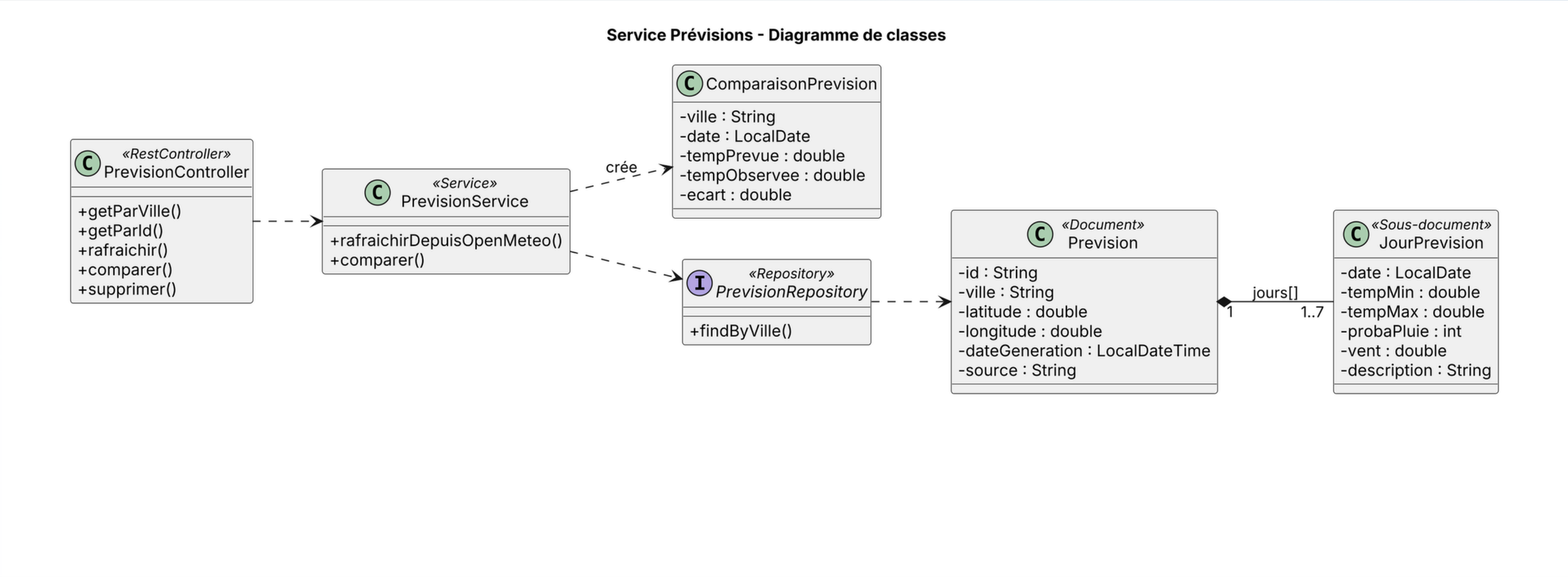
Si l'API externe est indisponible : on renvoie la dernière prévision en cache.

Pourquoi MongoDB ? Réponses JSON imbriquées (jours, heures) stockées telles quelles, schéma flexible.

MICROSERVICE 5 · MONGODB

Gestion des prévisions

Responsable : Amine Arfaoui



MICROSERVICE 6 · MONGODB

Gestion des notifications

Responsable : Mohamed Taher Elkhouja

Documents (modèle de données)

Notification : id, utilisateurId, alerteId, canal (EMAIL / PUSH), titre, message, statut (ENVOYEE / ECHEC / LUE), dateEnvoi

ModeleMessage : id, type, gabarit

Historique conservé par utilisateur

Contrat d'API REST

GET /api/notifications/me → 200 / 401

GET /api/notifications/{id} → 200 / 404

PATCH /api/notifications/{id}/lue → 200

POST /api/notifications/test → 201 (admin)

GET /api/modeles → 200

PUT /api/modeles/{id} → 200 / 404

Interactions

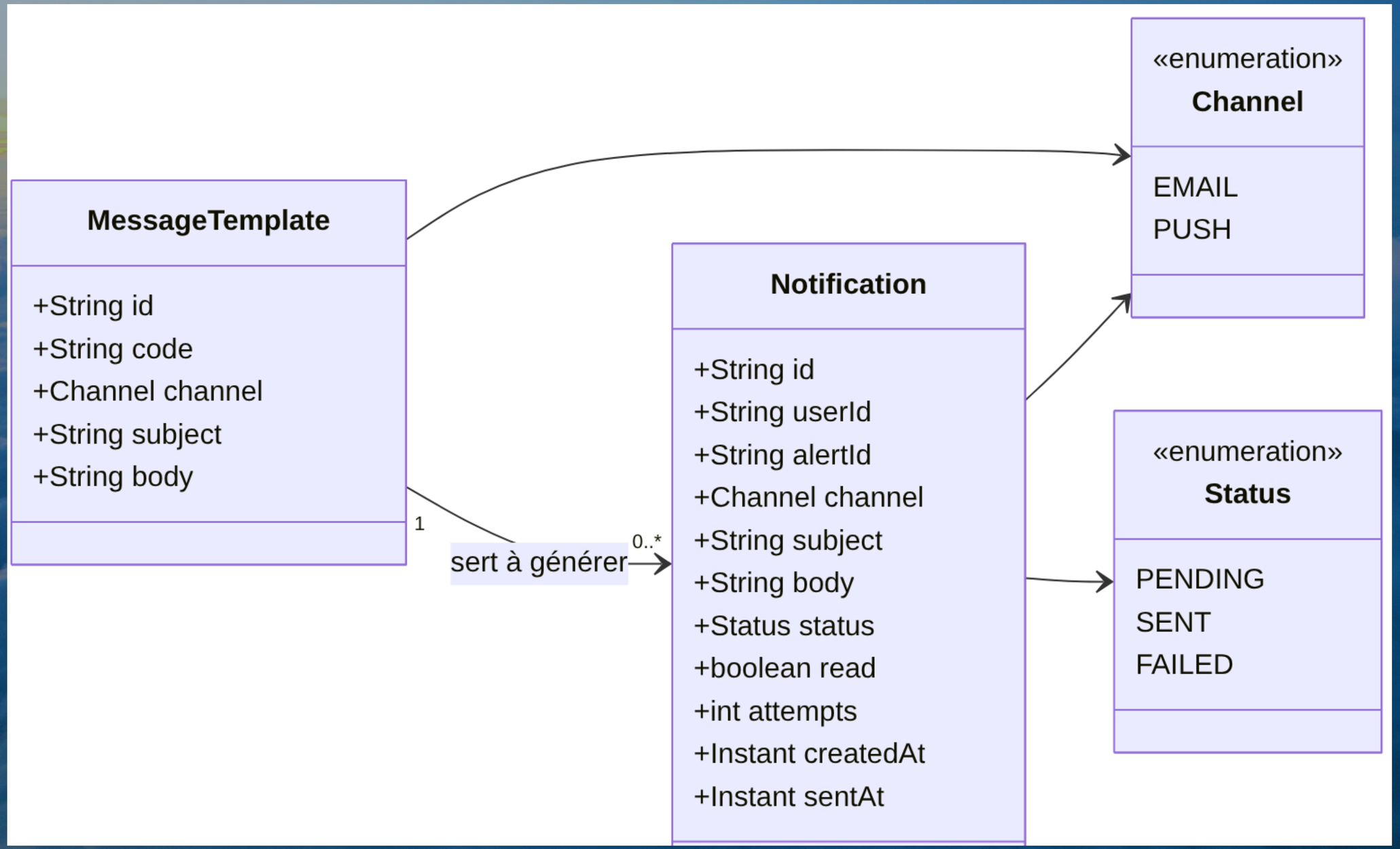
Consomme (RabbitMQ) : AlertTriggered

Appelle (OpenFeign) : Abonnements → abonnés de la région

Si Abonnements est indisponible : retry avec délai, puis dead-letter queue après plusieurs échecs.

Pourquoi MongoDB ? Historique volumineux, documents simples écrits souvent et lus par utilisateur.

Diagramme de classes – notification-service



MICROSERVICE COMMUN · TECHNOLOGIE AVANCÉE

Statistiques & tableau de bord

Développé par toute l'équipe · Présenté par Linda Nebily

Modèle de données

Indicateur : id, type, region, periode, valeur

Tendance : region, variable, evolution, dateCalcul

Données calculées à partir des relevés et des alertes

Contrat d'API REST

GET /api/stats/dashboard → 200

GET /api/stats/regions/{region} → 200 / 404

GET /api/stats/tendances?variable= → 200

GET /api/stats/prediction?ville= → 200

prediction = partie « technologie avancée »

Interactions

Consomme (RabbitMQ) : ObservationRecorded et AlertTriggered

Appelle (OpenFeign) : Prévisions → comparer prévu / observé

Si un service est indisponible : dernières statistiques calculées.

Techno avancée : module de prédiction en Python/FastAPI, ou assistant météo IA avec Spring AI.

METEOSPHERE

Technologies & organisation

Stack technique imposée et outils de l'équipe

Backend

MICROSERVICES

Spring Boot

Spring Cloud

Gateway · Eureka ·

Config OpenFeign

Frontend

INTERFACE & SÉCURITÉ

Angular

Keycloak (JWT)

Données

BASES & MESSAGING

PostgreSQL

MongoDB

RabbitMQ

Organisation

DÉPLOIEMENT & SUIVI

Docker Compose

Git

Jira (sprints)

METEOSPHERE

Planning & risques

01 Séance 4 : validation de la conception, l'architecture globale et le dépôt Git.

02 Sprint 1 : logique métier de chaque microservice, Gateway, Eureka, Config, Keycloak.

03 Sprint 2 : OpenFeign, RabbitMQ, Docker Compose, frontend et tests.

04 Risques : intégration RabbitMQ / Keycloak et coordination à 6 → réunions hebdomadaires, une branche par service.

METEOSPHERE

MERCI

Merci pour votre attention!

Équipe UniDev

Maha · Linda · Chaima
Ramzi · Amine · Med Taher